CHAPTER 1

DESIGNING THE AMAZON ONLINE SHOPPING SYSTEM

1 Getting Ready: The Amazon Online Shopping System . . . 2
2 Requirements for the Amazon Online Shopping System 4
3 Use Case Diagram for the Amazon Online Shopping System 5
4 Class Diagram for the Amazon Online Shopping System 8
5 Sequence Diagram for the Amazon Online Shopping System 22
6 Activity Diagram for the Amazon Online Shopping System 24
7 Code for the Amazon Online Shopping System 27
8 Summary: Designing the Amazon Online Shopping System 36
9 Key Insights: The Amazon Online Shopping System . . . 40

This chapter teaches you the design of the Amazon online shopping system, focusing on requirements, UML diagrams, and implementation code.

1 Getting Ready: The Amazon Online Shopping System

1.1 Problem definition

Amazon is an online retail platform that allows its users to buy from and sell products to other users. There are numerous product categories that customers can search and choose from. Products have reviews and ratings that can help customers make a buying decision. Each customer has a shopping cart where they can add products they wish to order. Once customers have added their favorite products to the cart, they can check out the cart and choose a payment method to place their orders. The customers are notified about their order and shipment statuses until the order is delivered. The system also allows users to add products they want to sell.

Figure 1.1: Users interacting with an online shopping system

1.2 Expectations from the interviewee

Numerous components are present in the Amazon online shopping system, each with specific constraints and requirements. The following provides an overview of some of the main things the interviewer will want to hear you discuss in more detail.

Discoverability

For an online shopping system like Amazon, discoverability is one of the key features that distinguishes it from others. You can ask the following questions to know more about the system:

- How will the buyer discover a product?
- How will the search surface result?

Cart and checkout

One of the most significant attributes of the online shopping system is the cart and checkout functionality that it provides to its customers. The interviewer expects the cart and checkout to behave in a certain way. You may ask the interviewer:

- How will the design adhere to known best practices while introducing innovative checkout semantics like “one-click purchase?”

Payment methods

The interviewer expects you to ask about the payment methods to determine the necessary requirements. You may ask the following questions:

- How will the users pay? Will they use credit cards, gift cards, and so on?
- How will the payment method work with the checkout process?

Product reviews and ratings

Product reviews and ratings help customers make informed decisions regarding what to buy. Knowing how to make this feature an effective part of the system is important. You can ask the interviewer:

- When can a user post a review and a rating?
- How are useful reviews tracked and less useful reviews deprioritized?

1.3 Design approach

We will design this online shopping system using the bottom-up design approach. For this purpose, we will follow the steps below:

- Identify and design the smallest components first, such as a product.
- Use these small components to design bigger components, like a product category that contains different products.
- Repeat the steps above until we design the whole system containing multiple product categories and users.

1.4 Design pattern

During an interview, it is always good practice to discuss the design patterns under which the Amazon online shopping system falls. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design an Amazon online shopping system? Please elaborate on your choice(s).

Let's explore the requirements of the online shopping system in the next lesson.

2 Requirements for the Amazon Online Shopping System

In this lesson, we'll list the requirements of our Amazon online shopping system. This is a crucial step as requirements define the scope of a problem. Therefore, getting them right from the interviewer and understanding them well will make the design of the rest of the system smooth and easy.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

2.1 Requirement collection

The requirements for the Amazon online shopping system design problem are defined below:

R1: A customer can be an authenticated user or a guest. An authenticated user has a registered Amazon online shopping system account, whereas a guest does not.
R2: An authenticated user should be able to buy, sell, and search the products via the product name or category. A guest is only able to search for products.

Column component type 'MxGraphWidget' not yet supported.

R3: A product can have multiple customer reviews and ratings.
R4: An authenticated user should be able to add, remove, or modify product items in their shopping cart. Then, the authenticated user can check out and buy the items.

Column component type 'MxGraphWidget' not yet supported.

R5: The authenticated user should be able to specify a shipping address when placing an order.
R6: Payment can be made through credit cards, electronic bank transfers, or cash on delivery.

Column component type 'MxGraphWidget' not yet supported.

R7: An order can be canceled only if it hasn't been shipped.
R8: Notifications are sent whenever there is a change in the order or shipping status.
R9: Shipment can be tracked to see the current status and the estimated arrival time for the order.
R10: An admin who can add, modify, or remove product categories and block users should exist.
R11: An account should store personal information such as name, email, password, and other relevant details for admins and authenticated users.

Column component type 'MxGraphWidget' not yet supported.

We've identified our requirements for the problem, and in the next lesson, we will define different use cases of our Amazon online shopping system.

3 Use Case Diagram for the Amazon Online Shopping System

Let's build the use case diagram of the Amazon problem and understand the relationship between its different components. First , we'll define the different elements of Amazon, followed by the complete use case diagram of the system.

3.1 System

Our system is the Amazon shopping system.

3.2 Actors

Now, we'll define the main actors of Amazon.

Primary actors

- **Authenticated user:** This actor can search for products, place or cancel orders, and add new products to sell.

- **Guest:** The guest actor can search for products, add items to a shopping cart, and update it. However, it must become a registered/authenticated member to place an order.

Secondary actors

- **Admin:** This can add, modify, or delete existing product categories.
- **System:** This is responsible for sending out notifications for orders and shipping updates.

3.3 Use cases

In this section, we will define Amazon's use cases. We have listed them according to their interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

Admin

- **Block account:** To block an account
- **Add/Modify/Delete product category:** To add, modify, or delete product categories

Authenticated user

- **Add/Modify/Delete product:** To add a new product or modify the details of any existing product that a customer themselves added for selling. A user can also delete a product that they added for selling
- **Search product:** To search for any particular product based on the given criteria (name or category)
- **Add item to shopping cart:** To add an item to a shopping cart
- **Update shopping cart:** To update the item quantities present in the shopping cart
- **Checkout shopping cart:** To check out from the shopping cart into the payment section
- **Add shipping address:** To add a shipping address
- **Add credit card:** To pay the order amount via credit card
- **Make payment:** To pay the order amount via credit card, electronic bank transfer, or cash on delivery
- **Manage shipment:** Manage shipment-related cases, such as sending an order for shipment, update shipment status, and sending notifications whenever shipment status changes.
- **Track shipment:** To track the shipment status.

Guest

- **Register account:** To register for an account on Amazon
- **Search product:** To search for any particular product based on the given criteria (name or category)

System

- **Send order notification:** To send a notification of an order after payment
- **Send shipment update notification:** To send a notification of any updates in the shipment

3.4 Relationships

This section describes the relationships between and among actors and their use cases.

Associations

The table below shows the association relationship between actors and their use cases.

Admin	**Authenticated User**	**Guest**	**System**
Block/Unblock account	Make payment	Register account	Send order notification
Add/modify product category	Add shipping address	Search product	Send shipment update notification
	Add/modify product		
	Search product		
	Add item to shopping cart		
	Update shopping cart		
	Checkout shopping cart		
	Mange shipment		
	Track shipment		

Include

When a user wants to make a payment, the system must verify the payment and then send a notification confirming the order.

- The "**Make Payment**" use case includes the "**Send Order Notification**" use case.

When a user searches for a product, the system allows them to search by category or name.

- The "**Search Product**" use case includes **s** earching for products by category or searching for products by name.

Extend

When a user chooses to make a payment, they can do so using a credit card, electronic bank transfer, or cash.

- The "**Make Payment"** use case encompasses payment options such as credit card, electronic bank transfer, andcash.

3.5 Use case diagram

Here's the use case diagram of Amazon:

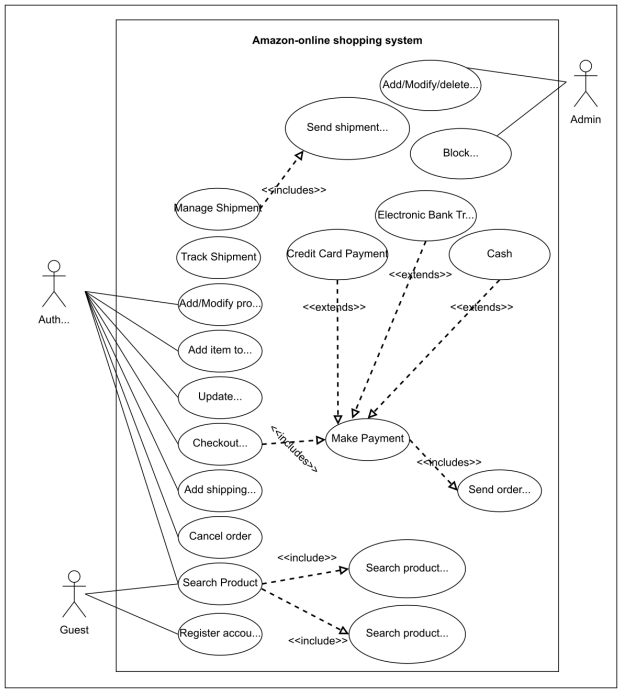


Figure 1.2: The use case diagram of the Amazon online shopping system

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Amazon Online Shopping System

In this lesson, we'll identify and design the classes, abstract classes, and interfaces based on the requirements we previously gathered from the interviewer in our Amazon shopping system.

4.1 Components of Amazon

As mentioned, we should design the Amazon online shopping system using a bottom-up approach.

Customer

The `Customer` abstract class refers to a user searching for a product on Amazon.
A customer can be one of the following:

1. Authenticated users
2. Guest users

The details of these are given below:

- The `AuthenticatedUser` class refers to an individual with a registered Amazon account.
- The `Guest` class refers to an individual without an account who can only search for and view the products on the Amazon website. However, they need to register for an account on Amazon to place an order.

The class diagram of all these is provided below:

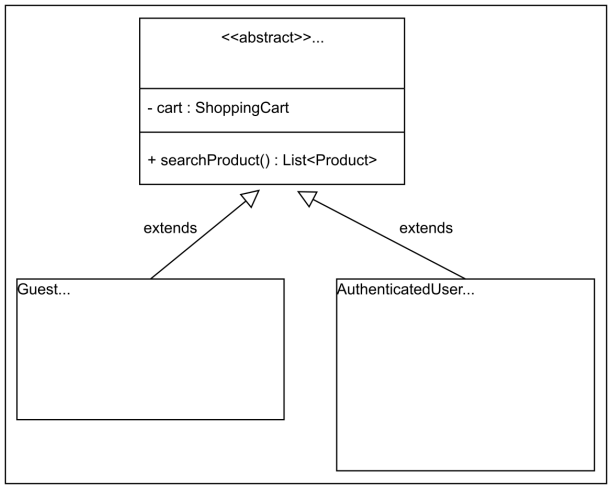


Figure 1.3: The class diagram of the Customer class and its child classes

R1: A customer can be an authenticated user or a guest. The authenticated user has a registered Amazon online shopping system account, whereas a guest does not.

Admin

The `Admin` class refers to an individual with a registered account on Amazon who can add, modify, or delete product categories and block users.

R10: An admin who can add or modify product categories and block users should exist.

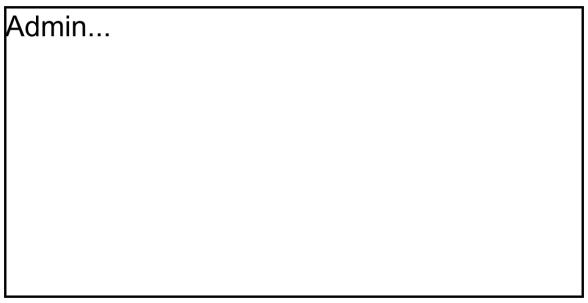


Figure 1.4: The class diagram of the Admin class

Account

The `Account` class accesses and showcases the personal details of the authenticated user and the admin, the two types of registered accounts available in the system. Users with an account can add and delete products and product reviews.

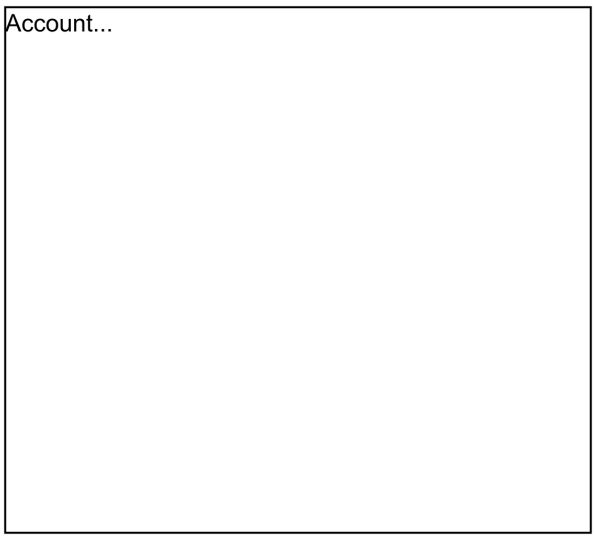


Figure 1.5: The class diagram of the Account class

R1: A customer can be an authenticated user or a guest. The authenticated user has a registered account on the Amazon online shopping system, whereas a guest doesn't have a registered account.
R11: An account should store personal information such as name, email, password, and other relevant details for admins and authenticated users.

Product

The `Product` class contains the details of a particular product on the Amazon shopping store. Each product falls under a specific category on Amazon and can have none, one, or more reviews.

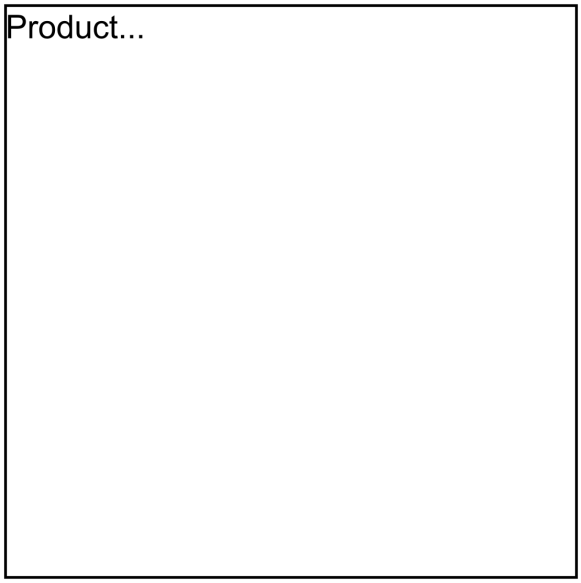


Figure 1.6: The class diagram of the Product class

R3: A product can have multiple reviews and ratings from multiple customers.

Product category

The `ProductCategory` class contains the names and descriptions of Amazon's various product categories and a reference to the list of products in each category.

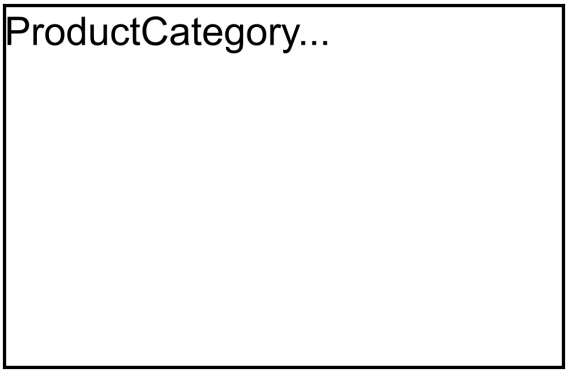


Figure 1.7: The class diagram of the ProductCategory class

R2: An authenticated user should be able to buy, sell, and search the products via the product name or category. A guest is only able to search for products.

Product review

The `ProductReview` class contains the rating and review attributes a registered user can use to add a review about a particular product.

Figure 1.8: The class diagram of the ProductReview class

R3: A product can have multiple reviews and ratings from multiple customers.

Search

The `Search` class will be an interface that contains the functionalities relating to searching for products.

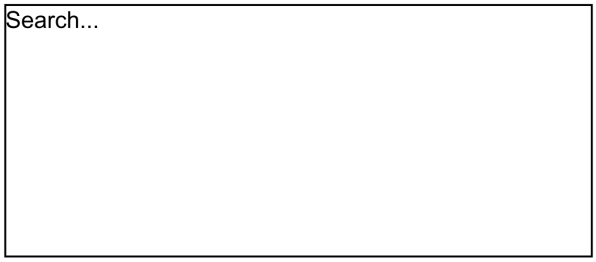


Figure 1.9: The class diagram of the Search interface

R2: An authenticated user should be able to buy, sell, and search the products via the product name or category. A guest is only able to search for products.

Cart item

The `CartItem` class refers to the items in the shopping cart. It will have price and quantity attributes, which can be modified using the `updateQuantity()` function.

R4: An authenticated user should be able to add, remove, or modify product items in their shopping cart. Then, the authenticated user can check out and buy the items.

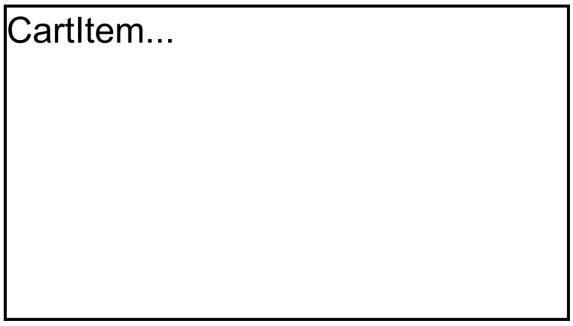


Figure 1.10: The class diagram of the CartItem class

Shopping cart

The `ShoppingCart` class contains the list of cart items, upon which actions such as adding items to the cart, removing items from the cart, and accessing all cart items can be performed.

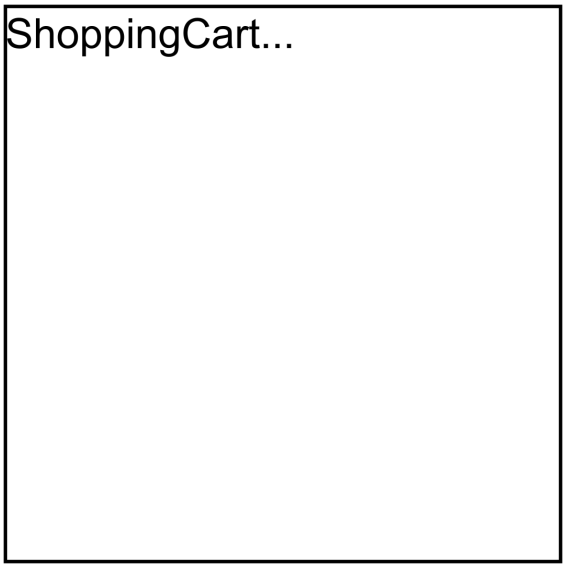


Figure 1.11: The class diagram of the ShoppingCart class

R4: An authenticated user should be able to add, remove, or modify product items in their shopping cart. Then, the authenticated user can check out and buy the items.

Order

The `Order` class refers to a particular customer's order and tracks its status, including the option of sending the order for shipment. It is also used to make the payment.

R7: An order can be canceled only if it hasn't been shipped.

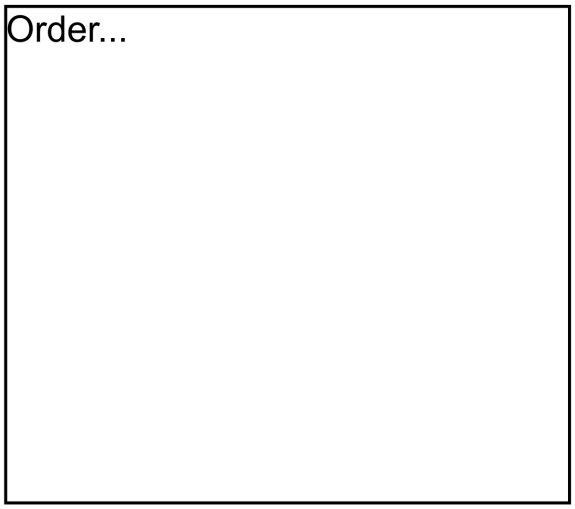


Figure 1.12: The class diagram of the Order class

Shipment

The `Shipment` class tracks the order's date, estimated arrival time, and shipment method. It will also be used to track the shipment status.

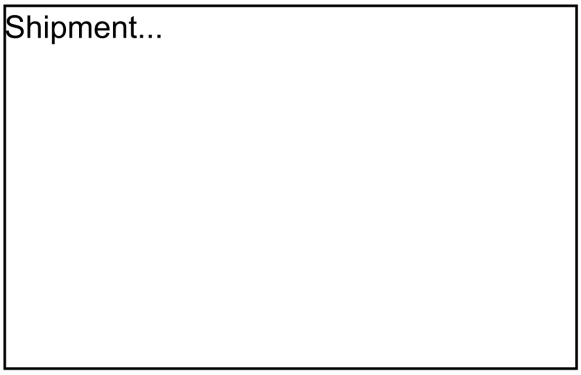


Figure 1.13: The class diagram of the Shipment class

R9: Shipment can be tracked to see the current status and the estimated arrival time for the order.

Payment

The `Payment` class will have three child classes: `CreditCard`, `ElectronicBankTransfer`, and `Cash`, as these are the three payment methods available to a customer on Amazon.

R6: Payment can be made through credit cards, electronic bank transfers, or cash on delivery.

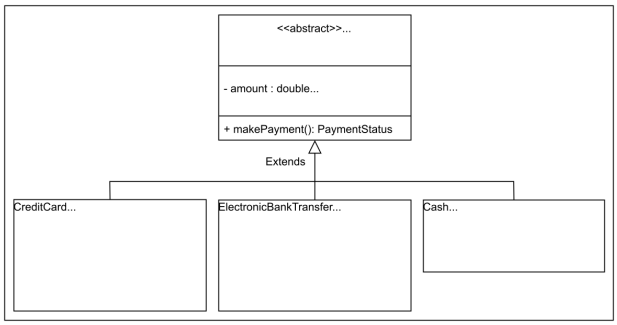


Figure 1.14: The class diagram of Payment and its child classes

Notification

The `Notification` class is responsible for sending order and shipment notifications to customers shown below in the class diagram:

> **Note:** Since the `Notification` class can be extended by adding various other options, we will implement it as an abstract class.

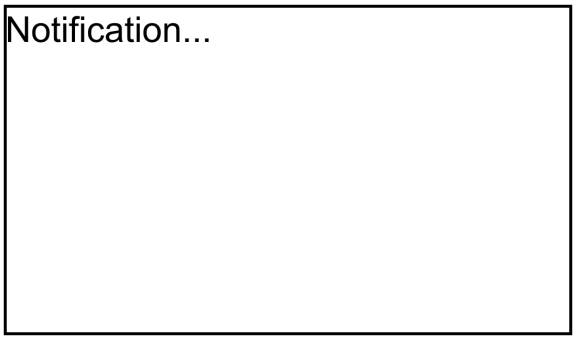


Figure 1.15: The class diagram of Notification and its child classes

R8: Notifications are sent whenever there is a change in the order or shipping status.

Enumerations

The following is the list of enumerations required in Amazon:

- `AccountStatus:` The account status tells about the user account status, whether it is active, inactive, or blocked.
- `PaymentStatus:` The payment status tells about the user account status, whether it is confirmed, declined, pending, or refunded.
- `OrderStatus:` The order status describes the status of a particular order of a customer, whether it is unshipped, pending, shipped, confirmed, or canceled.hipped, confirmed, or canceled.

- `ShipmentStatus:` The shipment status tells us about the status of an order's shipment, whether it is in a pending state, shipped state, delivered state, or on hold.

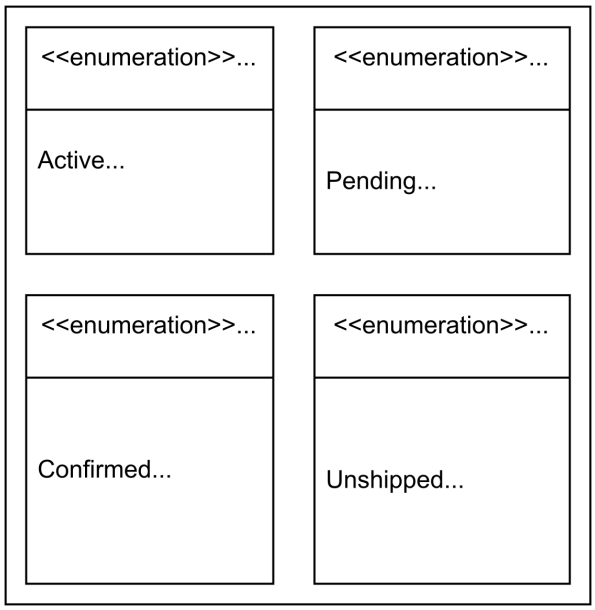


Figure 1.16: Enums in Amazon

Custom data type

To store a customer's location, we must create a custom data type, `Address`.

4.2 Relationship between the classes

We'll discuss the relationships between the classes defined in our Amazon shopping system above.

Association

The class diagram has the following association relationships:

- The `Notification` class has a *one-way association* with `Shipment` and `Order`.
- The `Product` class has a *two-way association* with `Account` and `CartItem` and a *one-way association* with `ProductCategory` and `ProductReview`.
- The `ShoppingCart` class has a *two-way association* with `CartItem`.
- The `Order` class has a *two-way association* with `Payment` and a *one-way association* with `Shipment` and `ShoppingCart`.

Figure 1.17: The class diagram of the Address custom data type

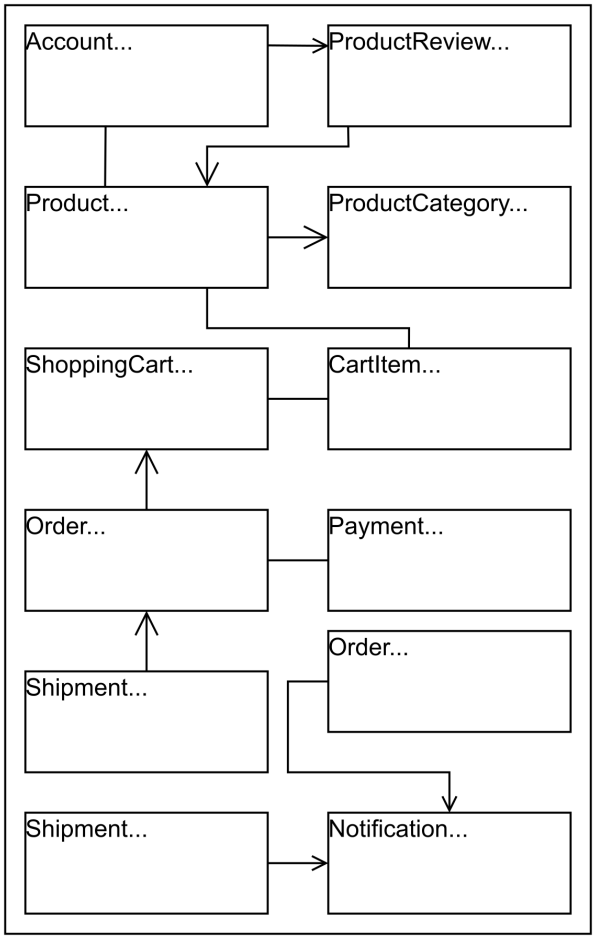


Figure 1.18: The association relationship between classes

Composition

The class diagram has the following composition relationships:

- The `Account` class is composed of the CreditCard and `ElectronicBankTransfer`.
- The `Admin` and `AuthenticatedUser` classes are composed of `Account`.
- The `AuthenticatedUser` class is composed of the `Order` class.
- The `Customer` class is composed of the `ShoppingCart` class.

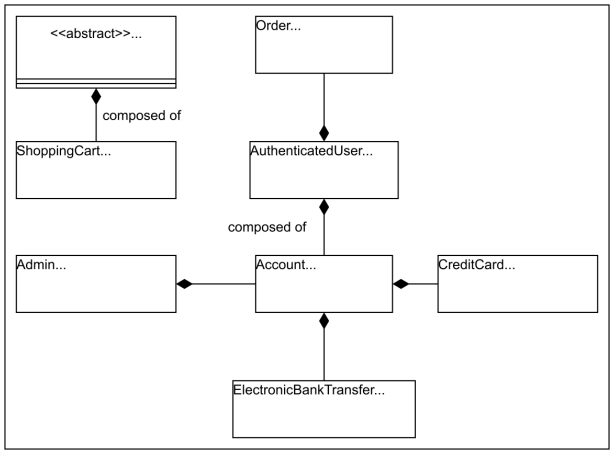


Figure 1.19: The composition relationship between classes

Aggregation

The following class show an aggregation relationship:

- The `Product` class has an aggregation relation with the Search class.

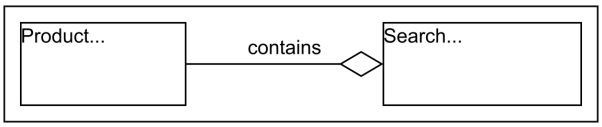


Figure 1.20: The aggregation relationship between classes

Inheritance

The following classes show an inheritance relationship:

- Both `Guest` and `AuthenticatedUser` extend the `Customer` class.
- Subclasses `Cash`, `ElectronicBankTransfer`, and `CreditCard` extend the `Payment` class.

  Note: The component section above has already discussed the inheritance relationship between classes.

4.3 Class diagram of Amazon

This section outlines the multiplicity (cardinality) relationships between the main classes in our Amazon online shopping system. We explain the allowed number of instances on each side for each relationship and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Here's the complete class diagram for Amazon:

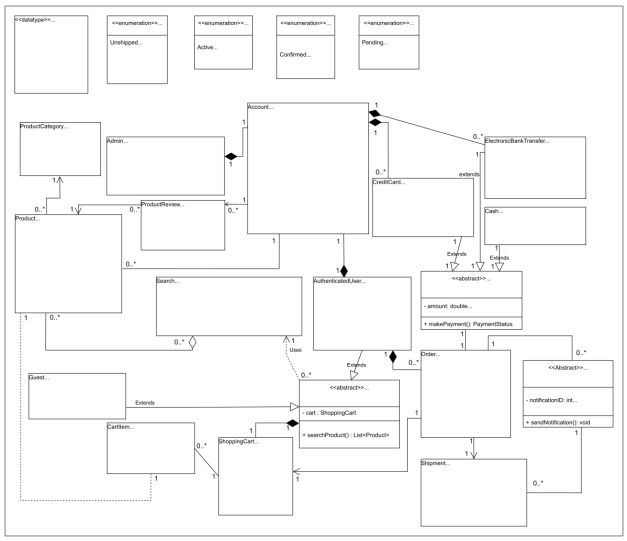


Figure 1.21: The class diagram of Amazon

4.4 Design pattern

The following design patterns have been used in the class diagram:

- We know that the system needs to create different types of accounts, such as `AuthenticatedUser` and `Guest` accounts. To model this behavior, we can use the Factory Method pattern.
- We know that a restaurant can accept different types of payments, such as `Cash`, `CreditCards`, and `ElectronicBankTransfers`. To model this behavior, we can use the Strategy pattern.

Source	Target	Multiplicity	Reason
Address	ProductCategory	1–0..*	One Address can be associated with many ProductCategory objects.
ProductCategory	Product	1–0..*	One ProductCategory can have many Products, but a Product belongs to exactly one ProductCategory.
Product	ProductReview	1–0..*	One Product can have many ProductReview objects.
ProductReview	AuthenticatedUser	1–1	One ProductReview is written by one AuthenticatedUser.
Admin	Account	1–0..*	One Admin can manage multiple Account objects.
Account	Order	1–0..*	One Account can place many Order objects.
Order	Shipment	1–0..*	One Order can be associated with many Shipment objects.
Order	PaymentStatus	1–1	One Order has exactly one PaymentStatus object.
ProductCategory	Admin	1–1	One ProductCategory is managed by one Admin.
ShoppingCart	Product	1–0..*	One ShoppingCart can contain multiple Product objects.
Guest	Product	1–0..*	One Guest can search many Product objects.
Search	Product	1–0..*	One Search operation can return multiple Product objects.
AuthenticatedUser	ShoppingCart	1–1	One AuthenticatedUser can have exactly one ShoppingCart.
ShoppingCart	Order	1–1	One ShoppingCart is associated with exactly one Order.

Table 1.1: Entity Relationships and Multiplicities in an E-Commerce System

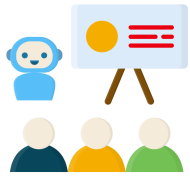
- We know that a `ShoppingCart` can contain multiple `Product` objects, and a `ProductCategory` can also contain multiple `Product` objects. To model this behavior, we can use the Composite pattern.

- When an order's status changes, we know that customers need to be notified. To model this behavior, we can use the Observer pattern.

- We know that various actions need to be performed on the `Order` and `ShoppingCart`, such as `placeOrder()`, `makePayment()`, and `addProduct()`. To model this behavior, we can use the Command pattern.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



4.6 Additional requirements

The interviewer can introduce some additional requirements in the Amazon shopping system, or they can ask some follow-up questions. Let 's see some examples of additional requirements:
Wish list: Only authenticated users can have a wish list, and they can add products to their wish list. Wish list items can be moved to the user's shopping cart. The `Wishlist` class has a two-way association with the `Product` class, as a wish list can contain multiple products, and products can belong to multiple wish lists, while it has an aggregation relationship with the `User` class, as a user can have a wish list, and a wish list cannot exist without a user.

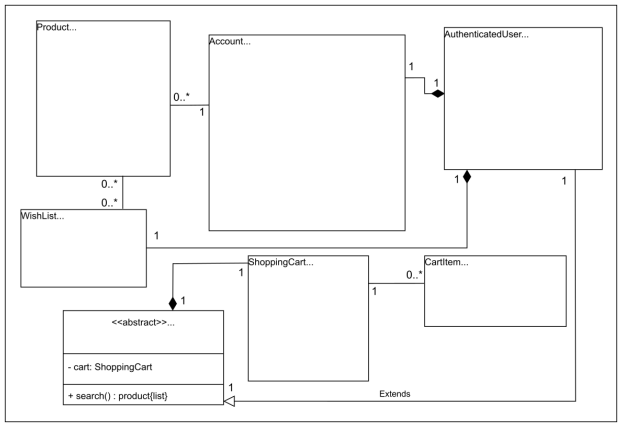


Figure 1.22: Adding Wishlist functionality

Discount: A discount will be applied to the payment depending on special events such as Christmas, Black Friday, etc. The `Payment` class has a one-way association with the `Discount` class, as a payment can receive a discount, but the discount is linked to the payment and does not exist independently.

We've completed the class diagram of the Amazon shopping system according to the requirements. Now , let's design the sequence diagram of the Amazon shopping system in the next lesson.

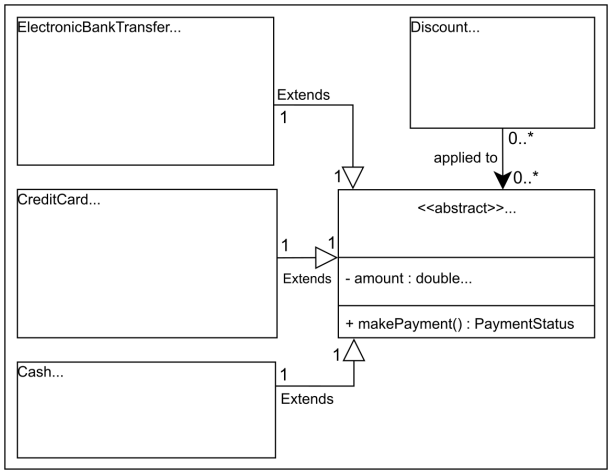


Figure 1.23: Relationship between the Discount class with the Payment class

5 Sequence Diagram for the Amazon Online Shopping System

Sequence diagrams are a great way to understand the interactions between different entities and objects in the system. There can be different sequence diagrams that we can create for the Amazon online shopping system. For this lesson, we'll create sequence diagrams for the following two interactions:

- **Checkout and payment:** The user checks out and pays for their order using a credit card.
- **Sequence challenge:** The user searches and adds an item to the shopping cart.

5.1 Checkout and payment

The sequence diagram for the order checkout and payment should have the following actors and objects that will interact with each other:

- **Actor:** `Customer`
- **Objects:** `ShoppingCart`, `Order`, `Item`, and `Payment`

Here are the steps in the order checkout and payment interaction:

1. The customer gets the shopping cart.
2. The customer then places the order for the requested items.
3. The items are verified for availability.
4. If the item quantity is less than the available count:

(a) The order status is updated.

(b) The customer is informed that the order is ready for payment.

(c) The customer initiates a payment against the order amount using their card.

(d) The payment is processed.

(e) The customer is informed of the updated payment status.

5. Else if the item quantity is greater than the available count:

   (a) The item in the order is unavailable.

   (b) The customer receives an error for the unavailable item.

Based on the order above, the sequence diagram below represents the checkout and payment process in an online shopping system.

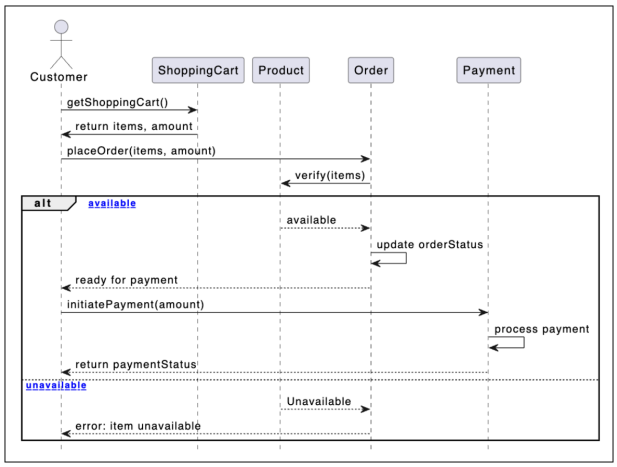


Figure 1.24: The sequence diagram for checkout and payment

5.2 Sequence challenge: Search and add items to the cart

You will help us complete a sequence diagram for searching and adding items to the cart. A skeleton of the search and add item sequence diagram is given below:

Notice that the arrows in the diagram above are numbered from 1 to 5. Below are the messages between the actor(s) and object(s). Can you rearrange the messages below in the correct order sequence they should appear in the skeleton of the sequence diagram provided above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can also click the "Show complete diagram" button below to view the complete sequence diagram for the search and add items to the cart interaction.

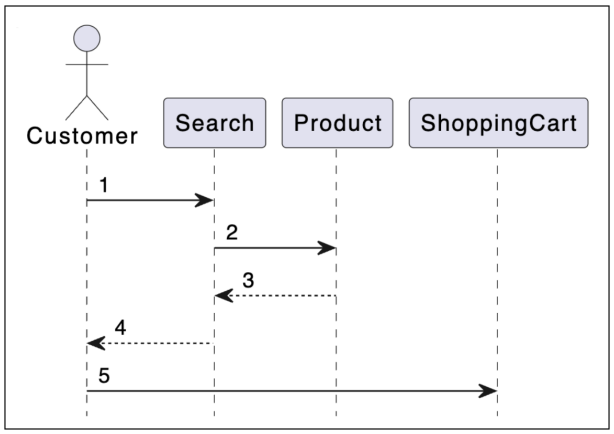


Figure 1.25: The sequence diagram for search and add items to the cart

Next, let's look at the activity diagrams for the online shopping system to understand the control flow of the system.

6 Activity Diagram for the Amazon Online Shopping System

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our Amazon online shopping system. In this lesson, we will create activity diagrams for the following two activities:

- A customer is buying a product.
- **Activity challenge:** The customer receives their order.

6.1 A customer buying a product

The following are the states and actions that will be involved in this activity diagram.

States

Initial state: The customer opens Amazon.
Final state: The customer proceeds with checkout.

Actions

The customer opens the website. They search or browse the website, select a product, and buy it.

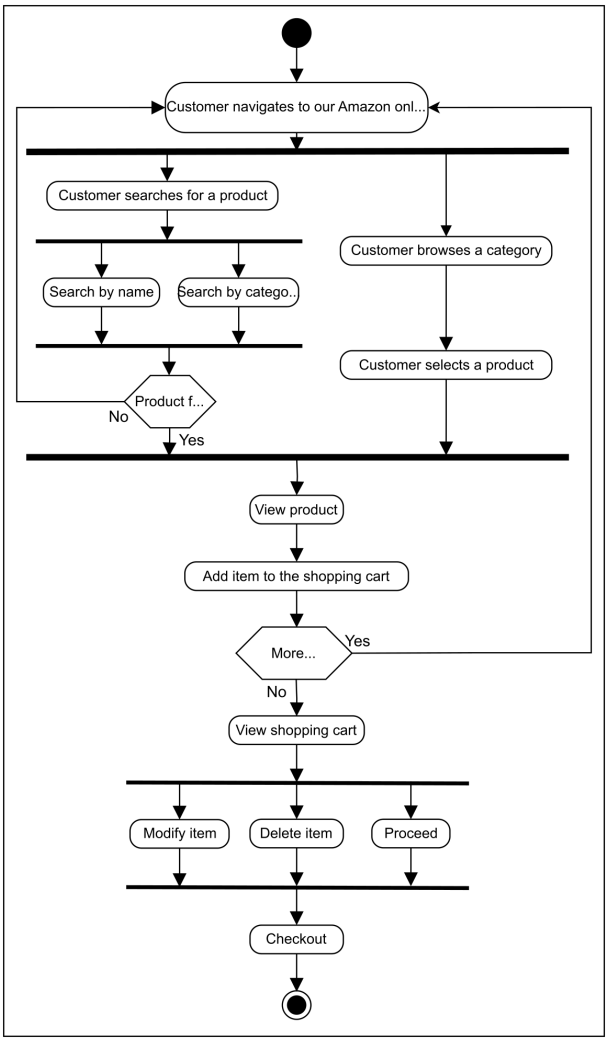


Figure 1.26: The activity diagram for a customer buying a product

6.2 Activity challenge: Customer order checkout and payment process

You'll help us create an activity diagram for a customer order checkout and payment process. In this activity, the customer checks out and selects a payment method—cash on delivery or credit card—before checkout.
A skeleton of the activity diagram is provided below.

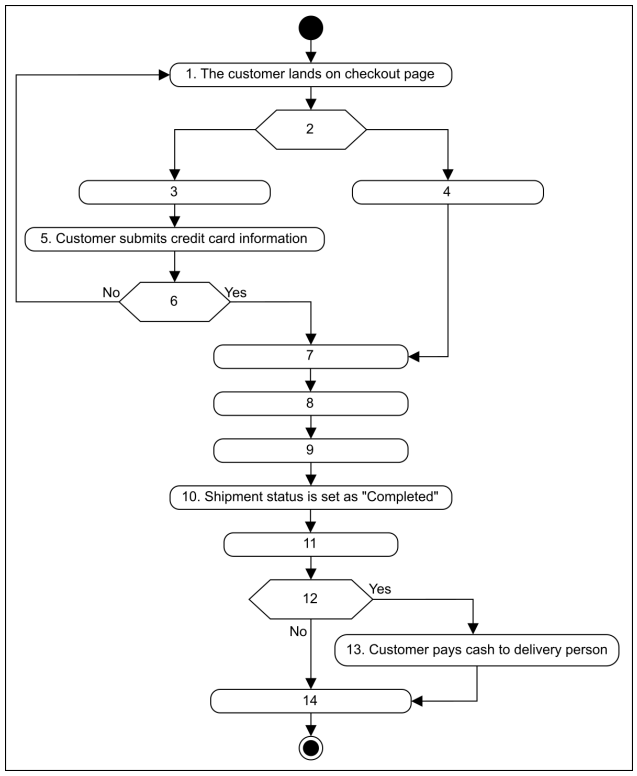


Figure 1.27: The activity diagram for a customer receiving their order

Notice that the actions in the diagram above are numbered from 1 to 14. The slots below represent the activities, and the arrows represent the flow from one activity to another.
Can you rearrange the slots below in the correct order they should appear in the activity diagram above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can click the "Show complete diagram" button below to see the complete activity diagram.

We've looked at some of the activity diagrams of our Amazon online shopping system. In the next lesson, we'll present the code for our designed classes in some of the most popular languages.

7 Code for the Amazon Online Shopping System

We've reviewed Amazon's different aspects and observed the attributes attached to the problem using various UML diagrams. Now , let's explore the more practical side of things, where we'll implement Amazon using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in Amazon:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Amazon Online Shopping System.

7.1 Amazon classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

Constants

The following code defines the various enums and custom data types being used in the Amazon design:

> **Note:** JavaScript does not support enumerations, so we will use the Object.freeze() method as an alternative that freezes an object and prevents further modifications.

Constant definitions

```
public class Address {
  private int zipCode;
  private String address;
  private String city;
  private String state;
  private String country;

  public String getFullAddress();
```

```
}

enum OrderStatus {
  UNSHIPPED,
  PENDING,
  SHIPPED,
  CONFIRMED,
  CANCELED,
  REFUNDED
}

enum AccountStatus {
  ACTIVE,
  INACTIVE,
  BLOCKED
}

enum ShipmentStatus {
  PENDING,
  SHIPPED,
  DELIVERED,
  ON_HOLD
}

enum PaymentStatus {
  CONFIRMED,
  DECLINED,
  PENDING,
  REFUNDED
}
```

Account

The `Account` class refers to an account of a customer on Amazon and contains a customer's personal details, such as their name, shipping address, credit card information, etc. It also provides authenticated users and admins with the functionality to add products, product reviews, and reset passwords. The definition of this class is given below:

The Account class

```
public class Account {
  private String userName;
  private String password;
  private String name;
  private List<Address> shippingAddress;
  private AccountStatus status;
  private String email;
  private String phone;

  private List<CreditCard> creditCards;
  private List<ElectronicBankTransfer> bankAccounts;
```

```
  public Address getShippingAddress();
  public boolean addProduct(Product product);
  public boolean addProductReview(ProductReview review, Product product);
  public boolean deleteProduct(Product product);
  public boolean deleteProductReview(ProductReview review, Product product);
  public boolean resetPassword();
}
```

Admin

The `Admin` class refers to a person from the administration of Amazon who can block users, add, modify, or delete product categories. The definition of this class is provided below:

Admin class

```
public class Admin {
  private Account account;
  public boolean blockUser(Account account);
  public boolean addNewProductCategory(ProductCategory category);
  public boolean modifyProductCategory(ProductCategory category);
  public boolean deleteProductCategory(ProductCategory category);
}
```

Customer

The `Customer` class is an abstract class with the `AuthenticatedUser` and `Guest` classes derived from it. The classes are defined below:

Customer and its child classes

```
// Customer is an abstract class
public abstract class Customer {
  private ShoppingCart cart;

  public abstract List<Product> searchproduct(String name);


}

public class Guest extends Customer {
  public boolean registerAccount();
  public abstract List<Product> searchproduct(String name);
}

public class AuthenticatedUser extends Customer {
  private Account account;
  private Order order;

  public OrderStatus placeOrder(Order order);
```

```
  public abstract List<Product> searchproduct(String name);
  public ShoppingCart getShoppingCart();
}
```

Product, product category, and product review

The following code defines the classes relating to a particular product:

The Product, ProductCategory, and ProductReview classes

```
public class Product {
  private String productId;
  private String name;
  private String description;
  private byte[] image;
  private double price;
  private ProductCategory category;
  private List<ProductReview> reviews;
  private int availableItemCount;
  private Account account;

  public int getAvailableCount();
  public int updateAvailableCount();
  public boolean updatePrice(double newPrice);
}

public class ProductCategory {
  private String name;
  private String description;
  private List<Product> products;
}

public class ProductReview {
  private int rating;
  private String review;
  private byte[] image;
  private AuthenticatedUser user;
}
```

Shopping cart and cart items

The `ShoppingCart` and CartItem classes add items to the shopping cart, update the quantities, and send the list of items to checkout. Both classes are defined below:

The ShoppingCart and CartItem classes

```
public class CartItem {
  private int quantity;
  private double price;
```

```
  public boolean updateQuantity(int quantity);
}

public class ShoppingCart {
  private int totalPrice;
  private List<Items> items;

  public boolean addItem(Item item);
  public boolean removeItem(Item item);
  public List<Item> getItems();
  public boolean checkout();
  public boolean verify();
  public double getTotalPrice();
}
```

Order

The `Order` class is responsible for making the payment, updating the order status, and sending the particular order for shipment. The definition of the class is given below:

The Order class

```
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class Order {
  private String orderNumber;
  private AuthenticatedUser cutomer;
  private Date orderDate;
  private OrderStatus status;
  private ShoppingCart shoppingCart;

  public Order(String orderNumber, OrderStatus status) {
    this.orderNumber = orderNumber;
    this.orderDate = new Date();
    this.status = status;

  }

  public boolean sendForShipment();
  public PaymentStatus makePayment(Payment payment);
  public boolean verify(CartItem item);
}
```

Shipment

The `Shipment` class keeps track of all the major details of the order's dispatch and creates the shipment record. The class is defined below:

The Shipment class

```
public class Shipment {
  private String shipmentNumber;
  private Date shipmentDate;
  private Date estimatedArrival;
  private String shipmentMethod;
  private ShipmentStatus status;

  public Shipment(String shipmentNumber, String shipmentMethod, Date
      estimatedArrival, ShipmentStatus status) {}
}
```

Payment

The `Payment` class is another abstract class with the `ElectronicBankTranfer`, `CreditCard`, and `Cash` classes as its child classes. This takes the `PaymentStatus` enum to keep track of the payment status. The definition of this class is provided below:

Payment and its child classes

```
// Payment is an abstract class
public abstract class Payment {
  private double amount;
  private Date timestamp;
  private PaymentStatus status;
  public abstract PaymentStatus makePayment();
}

public class CreditCard extends Payment {
  private String nameOnCard;
  private String cardNumber;
  private String billingAddress;
  private int code;
  public PaymentStatus makePayment();
}

public class ElectronicBankTransfer extends Payment {
  private String bankName;
  private String routingNumber;
  private String accountNumber;
  private String billingAddress;
  public PaymentStatus makePayment();
}

public class Cash extends Payment {
  private String billingAddress;
  public PaymentStatus makePayment();
}
```

Notification

The `Notification` class is responsible for sending notifications to customers about the order and shipment status. It is defined below:

Notification and its child classes

```
// Notification class
public class Notification {
  private int notificationId;
  private Date createdOn;
  private String content;

  public abstract boolean sendNotification(Account account);
}
```

Search

The `Search` class enables the search functionality based on the given criteria (name and category of product).

The Search class

```
public class Search {
  private HashMap<String, List<Product>> products = new HashMap<>();

  public void setProducts(Map<String, List<Product>> products);
  public List<Product> searchProductsByName(String name);
  public List<Product> searchProductsByCategory(String category);
  public Product getProductDetails(String productId);
}
```

7.2 Executable code: Amazon online shopping system

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Amazon Online Shopping System. The system's main driver code resides in `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

What does this code show

- **System initialization:** Initializes a guest user who browses the platform and registers an account. It also initializes an authenticated user (John) and creates a shopping cart and products to simulate the shopping experience.
- **Scenario 1 (Guest browses and registers):** A guest user browses the platform and registers a new account with a username, password, and email. This simulates a typical

scenario where new users create an account to start shopping.

- **Scenario 2 (Authenticated user adds items to cart and places an order):** The authenticated user, John, browses the available products, adds items (a laptop and a mouse) to his shopping cart, and proceeds to order. The cart is verified, and the order is placed for the first product in the cart, simulating the purchase process.
- **Scenario 3 (Order payment with credit card):** John makes the payment using a credit card for the order in Scenario 2. The payment is processed, and the shipment is prepared. The shipment details and payment status are displayed, showing the final transaction completion stages.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Amazon Online Shopping System

```
import java.util.*;

public class Driver {
  public static void main(String[] args) {
    System.out.println("========== Welcome to the Amazon Online Shopping System
        ==========
");

    // Scenario 1: Guest Browses and Registers
    System.out.println("==== Scenario 1: Guest Browses and Registers ====");
    Guest guest = new Guest();
    System.out.println(" Guest browses the platform...");
    guest.registerAccount("john_doe", "password123", "john@example.com");

    // Scenario 2: Authenticated User Adds Items to Cart and Places an Order
    System.out.println("
==== Scenario 2: Authenticated User Adds Items to Cart and Places an Order ====");

    // 2.1. Authenticated User Setup
    Account johnAccount = new Account();
    johnAccount.setStatus(AccountStatus.ACTIVE);
    AuthenticatedUser john = new AuthenticatedUser();

    // 2.2. Create Product Catalog
    Search search = new Search();
    ProductCategory electronics = new ProductCategory("Electronics", "Devices and
        gadgets");
    Product laptop = new Product("P001", "Gaming Laptop", "Powerful 16GB RAM laptop
        ", 1500.00, electronics);
    Product mouse = new Product("P002", "Wireless Mouse", "Ergonomic wireless mouse
        ", 25.00, electronics);

    // Manually populate product catalog
    List<Product> electronicsList = new ArrayList<>();
```

```
    electronicsList.add(laptop);
    electronicsList.add(mouse);
    Map<String, List<Product>> productMap = new HashMap<>();
    productMap.put("electronics", electronicsList);
    search.setProducts(productMap);

    System.out.println(" Products available in 'Electronics':");
    for (Product p : search.searchProductsByCategory("electronics")) {
      System.out.println(" - " + p.getName() + ": $" + p.getPrice());
    }

    // 2.3. User adds items to shopping cart
    System.out.println("
 User adds items to shopping cart:");
    john.addItem(laptop, 1);
    john.addItem(mouse, 2);

    // 2.4. Show cart summary
    System.out.println("
 Shopping Cart Summary:");
    ShoppingCart cart = john.getShoppingCart();
    for (CartItem item : cart.getItems()) {
      System.out.println(" - " + item.getProduct().getName() + " x " + item.
          getQuantity() + " = $" + item.getPrice());
    }
    System.out.println("Total: $" + cart.getTotalPrice());

    // 2.5. Verify cart and place order
    if (cart.verify()) {
      System.out.println("
 Cart verified. Proceeding to place order...");
    }
    CartItem itemToOrder = cart.getItems().get(0);
    OrderStatus status = john.placeOrder(itemToOrder, itemToOrder.getPrice());

    // Scenario 3: Order Payment with Credit Card
    System.out.println("
==== Scenario 3: Order Payment with Credit Card ====");
    // 3.1. Process payment using CreditCard
    Payment payment = new CreditCard(itemToOrder.getPrice(), "John Doe", "
        1234567812345678", "123 Street, NY", 123);

    // 3.2. Order and Shipment
    Order order = new Order("ORD123", status);
    PaymentStatus paymentStatus = order.makePayment(payment);

    Shipment shipment = new Shipment("SHIP123", "FedEx", new Date(System.
        currentTimeMillis() + 3 * 86400000), ShipmentStatus.PENDING);

    // 3.3. Display Shipment and Payment Status
    // System.out.println("
 Shipment Details:");
    // for (ShipmentLog log : shipment.getShipmentLogs()) {
    // System.out.println(" - Status: " + log.getStatus() + ", Logged on: " + log.
        getCreationDate());
    // }
```

```
    System.out.println("
 Payment Status: " + paymentStatus);
    System.out.println(" Order Status: " + status);

    System.out.println("
 Thank you for shopping with us!");
    System.out.println("=====================================================");
  }
}
```

7.3 Wrapping up

In this chapter, we've explored Amazon's complete design, including how it is visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing the Amazon Online Shopping System

Now that you've completed the Amazon online shopping system case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

This section outlines the primary functional requirements that guided the design of the online shopping system.

1. The system must support registered `AuthenticatedUser` accounts and non-registered `Guest` users.
2. Authenticated users can buy, sell, and search for products, while guests can only search.
3. Products can have multiple reviews and ratings from authenticated users.
4. Authenticated users must have a shopping cart to add, remove, and modify items before checkout.
5. Users must be able to specify a shipping address for their orders.

6. The system must support multiple payment methods, including credit cards, electronic bank transfers, and cash on delivery.
7. Orders can be canceled only if they have not yet been shipped.
8. The system must notify users about order and shipment status changes.
9. Users must be able to track the shipment status and estimated arrival of their orders.
10. An `Admin` role must exist to manage product categories and block users.
11. The system must store personal information for registered accounts, including name, email, and password.

8.2 System actors

The following actors interact with the online shopping system in various capacities.

- **Authenticated User:** A registered user who can search for products, manage their shopping cart, place and cancel orders, and sell their products.
- **Guest:** A non-registered user who can search for and view products but must register to make a purchase.
- **Admin:** A user with special privileges to manage product categories and block user accounts.

8.3 Important classes and relationships

The system's most important classes, descriptions, and key relationships are listed below:

8.4 Design highlights

This section covers the overall design methodology, the core principles applied, and the specific patterns used to structure the system.

Design approach

The system was designed using a bottom-up approach. This methodology identifies and designs the smallest, most granular components, such as the `Product` and `ProductReview` classes. These smaller components are then assembled into larger, more complex components like `ProductCategory` and `ShoppingCart`, continuing until the entire system is built.

Design principles

The design of the Amazon system effectively demonstrates the five SOLID principles, leading to a more maintainable, scalable, and robust architecture.

Class	Description	Important Relationships
Customer	An abstract class representing a user of the system.	Parent of AuthenticatedUser and Guest (inheritance).
AuthenticatedUser	Represents a registered user with a full set of permissions.	Composed of Account and Order (composition).
Account	Stores personal details, credentials, and payment information for a registered user or admin.	Composed of CreditCard and ElectronicBankTransfer.
Product	Contains all information about an item for sale, including its category, price, and reviews.	Associated with ProductCategory and ProductReview.
ShoppingCart	Holds a collection of CartItem objects that a user intends to purchase.	Associated with CartItem (two-way).
Order	Represents a customer's request to purchase items, tracking status from placement to completion.	Associated with Payment and Shipment.
Payment	An abstract class for processing payments.	Parent of CreditCard, ElectronicBankTransfer, and Cash.
Shipment	Tracks the shipping details and status of an order.	Associated with Order.
Notification	An abstract class responsible for sending notifications to users.	Associated with Order and Shipment.
Search	An interface that defines the contract for searching products by name or category.	Aggregated by Product (aggregation).

Table 1.2: E-Commerce System Classes and Their Relationships

- **Single Responsibility principle (SRP):** Each class is designed to have one primary responsibility. For instance, the `Order` class manages order data and status, the `Payment` class handles payment processing, and the `Notification` class is solely responsible for sending notifications.

- **Open/Closed principle (OCP):** The system is open to extension but closed for modification. This is evident in the use of abstract classes like `Payment` and `Notification`. New payment methods or notification types can be added by creating new subclasses without altering the existing, tested code.

- **Liskov Substitution principle (LSP):** Subtypes can be substituted for their base types without altering the program's correctness. For example, the system can use a `Customer` object to function correctly, whether the concrete instance is an `AuthenticatedUser` or a `Guest`.

- **Interface Segregation principle (ISP):** The `Search` interface is a clear example, providing only the methods relevant to searching. This ensures that client classes do

not depend on methods they don't use, preventing bloated interfaces.

- **Dependency Inversion principle (DIP):** The system depends on abstractions rather than concrete implementations. The `Order` class interacts with the abstract `Payment` class, not directly with a specific `CreditCard` or `Cash` class, which decouples the components.

Design patterns

The system strategically employs several design patterns to solve common object-oriented design problems.

- **Factory Method pattern:** Used to create different types of accounts (`AuthenticatedUser`, `Guest`) without exposing the instantiation logic to the client.
- **Strategy pattern:** Implemented to handle different payment methods (`CreditCard`, `Cash`), allowing the payment algorithm to be selected at runtime.
- **Composite pattern:** Used to treat individual `Product` objects and collections of products (`ProductCategory`, `ShoppingCart`) uniformly.
- **Observer pattern:** Applied to notify customers automatically whenever their `Order` or `Shipment` status changes.
- **Command pattern:** Used to encapsulate actions on objects like `Order` and `ShoppingCart` as distinct objects, enabling operations like `placeOrder()` and `addProduct()`.

8.5 Object interactions

Collaborating between key objects to fulfill user requests drives the system's dynamic behavior.

- **Checkout process:** When a `Customer` decides to check out, they interact with their `ShoppingCart`. The `ShoppingCart` data is used to create an `Order` object. The `Order`, which then verifies the availability of each `Product`.
- **Payment processing:** Once the order is verified, the `Customer` initiates payment through the `Order` object. The `Order` object delegates the processing to a concrete `Payment` object (e.g., `CreditCard`). The `Payment` object processes the transaction and returns a status, which the `Order` object then updates.
- **Handling unavailable items:** If a `Product` is unavailable during verification, the `Order` object flags it and sends an error message back to the `Customer`, preventing the process from proceeding to payment.

8.6 System workflow

The primary workflow involves a customer finding and purchasing a product, which unfolds through several steps.

- **Product discovery:** A `Customer` Guest or Authenticated) starts by navigating to the home page. They can use the search functionality (by name or category) or browse `ProductCategory` listings to find items.
- **Adding to cart:** Once a `Product` is selected, the `Customer` can view its details. An `AuthenticatedUser` can then add the item to their `ShoppingCart`. They can continue adding more products or proceed to view the cart.
- **Cart management and checkout:** In the `ShoppingCart`, the user can modify quantities, delete items, or proceed to checkout.
- **Finalizing the order:** The checkout process converts the `ShoppingCart` into an `Order`, where the user confirms shipping details and makes a payment, completing the purchase life cycle.

9 Key Insights: The Amazon Online Shopping System

Great job reaching this stage with your Amazon online shopping system design! This lesson will help you level up by sharing clear, actionable tips to strengthen your design, avoid common pitfalls, and build confidence for tackling tough interview questions.

9.1 Common mistakes

Many candidates overlook critical details when designing an online shopping system like Amazon. Here are some frequent mistakes that can undermine an otherwise good solution:

- **Missing guest user limitations:** Often, candidates forget to differentiate guest and authenticated user permissions. For example, allowing guests to place an order violates *R2,* which restricts guests to only searching for products. Always establish a clear inheritance hierarchy, i.e., model both `Guest` and `AuthenticatedUser` as child classes of the `Customer` abstract class. Then, enforce constraints in methods, for example, `Guest` should not have `placeOrder()` implemented, while `AuthenticatedUser` does. Add conditional logic in the checkout sequence to reject orders from guests and prompt them to register first.
- **Inadequate cart management:** Failing to handle edge cases like item unavailability during checkout. The sequence diagram shows the system must check item availability and gracefully handle insufficient stock. Design the `ShoppingCart` and `CartItem` classes to include checks for item availability during checkout. Use the `Order` class's `placeOrder()` or `makePayment()` methods to verify stock before confirming an order.

- **Unclear notification flow:** Overlooking that the system must send notifications for both order and shipment status changes, which involves the `Notification` class's association with `OrderLog` and `ShipmentLog`. Use the `Notification` abstract class and connect it with `OrderLog` and `ShipmentLog` through association relationships. Always design an `EmailNotification` or `SMSNotification` subclass that automatically triggers on relevant `OrderStatus` or `ShipmentStatus` changes. Show this explicitly in your class and sequence diagrams to clarify the interaction.

- **Generic search implementation:** Implementing a simple search functionality is not enough. Interviewees sometimes fail to elaborate on the `Search` interface and its implementation by the `Catalog` class, along with the specific search methods like `searchProductsByName()` and `searchProductsByCategory()`. This detail highlights how product discoverability (a key feature) is achieved.

- **Ignoring payment method variations:** Many forget to model payment method subclasses (`CreditCard`, `ElectronicBankTransfer`, `Cash`) correctly, missing the use of the Factory or Strategy pattern for dynamic payment handling. Apply the Factory pattern to instantiate the correct `Payment` subclass (`CreditCard`, `ElectronicBankTransfer`, or `Cash`). Use the Strategy pattern if dynamic behavior is needed, such as applying different discount calculations.

9.2 Key interview tips

Candidates who present their Amazon design clearly and handle follow-up questions confidently often keep a few practical strategies in mind:

Tip	**Explanation**
Clarify user roles early.	Start by explaining how Guest and AuthenticatedUser differ in the actions they can perform. This sets up the class and use case design logically.
Highlight design patterns.	Show where you'll use patterns like Factory (for order creation) and Strategy (for applying discounts). Interviewers appreciate this level of detail.
Show class relationships.	Use clear language to describe associations, composition, and inheritance. For example, Customer → Guest/AuthenticatedUser.
Discuss edge cases proactively.	Explain scenarios like order cancellation, refund handling, and stock checks during checkout. This demonstrates depth of understanding.
Keep the sequence flow logical.	When describing flows like checkout, walk step-by-step through the sequence: cart → order → payment → shipment → notification.

Table 1.3: Design and Explanation Tips for an E-Commerce System

9.3 Test your understanding

These tricky questions will help you assess your grasp of the key elements of the Amazon online shopping system design.

The Amazon Online Shopping System

Question 1: Which relationship correctly describes `Customer` and `Guest` in the class diagram?

- Aggregation
- Composition
- **[CORRECT]** Inheritance
  Explanation: The `Guest` class extends the `Customer` class, so it's inheritance.
- Association

Question 2: What must happen first if a guest adds items to the cart but wants to place an order?

- **[CORRECT]** Register as an authenticated user.
  Explanation: A guest must register first to place an order.
- Use one-click purchase.
- Proceed to checkout directly.
- Add a shipping address.

Question 3: Why is the `Notification` class abstract?

- It doesn't send notifications.
- It updates the catalog.
- It stores reviews.
- **[CORRECT]** Concrete types must inherit it.
  Explanation: It can be extended to `EmailNotification` or `SMSNotification`.

Question 4: Which class is responsible for storing multiple shipping addresses for a user?

- `Customer`
- `Order`
- **[CORRECT]** `Account`
  Explanation: The `Account` class contains user's shipping addresses.
- `ShoppingCart`

Question 5: How is the relationship between the `AuthenticatedUser` and the `Guest` class modeled?

- Composition
- **[CORRECT]** Generalization
  Explanation: `AuthenticatedUser` generalizes `Guest`, i.e., an authenticated user inherits guest capabilities plus more.
- Aggregation
- Association

9.4 Related case studies

If you found the Amazon online shopping system valuable, you'll benefit from these related design case studies too:

- **Designing a movie ticket booking system:** Similar checkout, payment, and seat availability checks help strengthen your edge-case handling.
- **Designing a hotel management system:** Covers booking, payment, user roles, and notification flows, which sharpens system state management.
- **Designing the eBay auction system:** Explore similar buyer–seller flows, but with auction-specific features like bidding windows and time-bound listings.
- **Designing the Walmart online grocery system:** Reinforces cart, payment, delivery flows, and inventory constraints for perishable goods.
- **Designing the Uber ride sharing system:** Focuses on live status tracking, dynamic matching, and multi-step payments similar to Amazon's shipment and order status.